2 Introducing Classes

Chapter Objectives

2.1 Constructor and destructor functions

2.2 Constructors that take parameters

2.3 Introducing inheritance

2.4 Object pointers

2.5 Classes, structures, and unions are related

2.6 In-line functions

2.7 Automatic in-lining

THIS chapter introduces classes and objects. Several important topics are covered that relate to virtually all aspects of C++ programming, so a careful reading is advised.

Review Skills Check

Before proceeding, you should be able to correctly answer the following questions and do the exercises.

1. Write a program that uses C++-style I/O to prompt the user for a string and then display its length.

2. Create a class that holds name and address information. Store all the information in character strings that are private members of the class. Include a public function that stores the name and address. Also include a public function that displays the name and address. (Call these functions **store()** and **display()**.)

3. Create an overloaded **rotate()** function that left-rotates the bits in its argument and returns the result. Overload it so it accepts **int**s and **long**s. (A rotate is similar to a shift except that the bit shifted off one end is shifted onto the other end.)

4. What is wrong with the following fragment?

```
#include <iostream>
using namespace std;

class myclass
{
    int i;
public:
.
.
.
};

int main()
{
    myclass ob;
    ob.i = 10;
.
.
.
}
```

2.1 CONSTRUCTOR AND DESTRUCTOR FUNCTIONS

If you have been writing programs for very long, you know that it is common for parts of your program to require initialization. The need for initialization is even more common when you are working with objects. In fact, when applied to real problems, virtually every object you create will require some sort of initialization. To address this situation, C++ allows a *constructor function* to be included in a class declaration. A class's constructor is called each time an object of that class is created. Thus, any initialization that needs to be performed on an object can be done automatically by the constructor function.

A constructor function has the same name as the class of which it is a part and has no return type. For example, here is a short class that contains a constructor function:

```
#include <iostream>
using namespace std;

class myclass
{
    int a;
public:
    myclass(); // constructor
    void show();
};

myclass::myclass()
{
    cout << "In constructor\n";
    a = 10;
}

void myclass::show()
{
    cout << a;
}

int main()
{
    myclass ob;

    ob.show();

    return 0;
}
```

In this simple example, the value of **a** is initialized by the constructor **myclass()**. The constructor is called when the object is created. An object is created when that object's declaration statement is executed. It is important to understand that in C++, a variable declaration statement is an "action statement." When you are programming in C, it is easy to think of declaration statements as simply establishing variables. However, in C++, because an object might have a constructor, a variable declaration statement may, in fact, cause a considerable number of actions to occur.

Notice how **myclass()** is defined. As stated, it has no return type. According to the C++ formal syntax rules, it is illegal for a constructor to have a return type.

For global objects, an object's constructor is called once, when the program first begins execution. For local objects, the constructor is called each time the declaration statement is executed.

The complement of a constructor is the *destructor*. This function is called when an object is destroyed. When you are working with objects, it is common to have to perform some actions when an object is destroyed. For example, an object that allocates memory when it is created will want to free that memory when it is destroyed. The name of destructor is the name of its class, preceded by a $\sim$. For example, this class contains a destructor function:

```
#include <iostream>
using namespace std;

class myclass
{
    int a;
public:
    myclass(); // constructor
    ~myclass(); // destructor
};

myclass::myclass()
{
    cout << "In constructor\n";
    a = 10;
}

myclass::~myclass()
{
    cout << "Destructing...\n";
}

void myclass::show()
{
    cout << a << "\n";
}

int main()
{
    myclass ob;

    ob.show();

    return 0;
}
```

A class's destructor is called when an object is destroyed. Local objects are destroyed when they go out of scope. Global objects are destroyed when the program ends.
It is not possible to take the address of either a constructor or a destructor.
Note: *Technically, a constructor or a destructor can perform any type of operation. The code within these functions does not have to initialize or reset anything related to the class for which they are defined. For example, a constructor for the preceding examples could have computed pi to 100 places. However, having a constructor or destructor perform actions not directly related to the initialization or orderly destruction of an object makes for very poor programming style and should be avoided.*

EXAMPLES

1. You should recall that the **stack** class created in Chapter 1 required an initialization function to set the stack index variable. This is precisely the sort of operation that a constructor function was designed to perform . Here is an improved version of the **stack** class that uses a constructor to automatically initialize a stack object when it is created:

```
#include <iostream>
using namespace std;

#define SIZE 10

// Declare a stack class for characters.
class stack
{
    char stck[SIZE]; // holds the stack
    int tos; // index of top of stack
public:
    stack(); // constructor
    void push(char ch); // push character on stack
    char pop(); // pop character from stack
};

// Initialize the stack.
stack::stack()
{
    cout << "Constructing a stack\n";
    tos = 0;
}

// Push a character
void stack::push(char ch)
{
    if(tos==SIZE)
    {
        cout << "Stack is full\n";
        return;
    }
    stck[tos] = ch;
    tos++;
}

// Pop a character
char stack::pop()
{
   if(tos==0)
   {
        cout << "Stack is empty\n";
        return 0; // return null on empty stack
    }
    tos--;
    return stck[tos];
}

int main()
{
    // Create two stacks that are automatically initialized.
    stack s1, s2;
```

```
    int i;

    s1.push('a');
    s2.push('x');
    s1.push('b');
    s2.push('y');
    s1.push('c');
    s2.push('z');

    for(i=0; i<3; i++)
        cout << "Pop s1: " << s1.pop() << "\n";
    for(i=0; i<3; i++)
        cout << "Pop s2: " << s2.pop() << "\n";

    return 0;
}
```

As you can see, now the initialization task is performed automatically by the constructor function rather than by a separate function that must be explicitly called by the program. This is an important point. When an initialization is performed automatically when an object is created, it eliminates any prospect that, by error, the initialization will not be performed. This is another way that objects help reduce program complexity. You, as the programmer, don't need to worry about initialization-it is performed automatically when the object is brought into existence.

2. Here is an example that shows the need for both a constructor and a destructor function. It creates a simple string class, called **strtype**, that contains a string and its length. When a **strtype** object is created, memory is allocated to hold the string and its initial length is set to 0. When **strtype** object is destroyed, that memory is released.

```
#include <iostream>
#include <cstring>
#include <cstdlib>
using namespace std;

#define SIZE 25
class strtype
{
    char *p;
    int len;
public:
    strtype(); // constructor
    ~strtype(); // destructor
    void set(char *ptr);
    void show();
};

// Initialize a string object.
strtype::strtype()
{
    p = (char *) malloc(SIZE);
    if(!p)
    {
```

```
        cout << "Allocation error\n";
        exit(1);
    }
    *p = '\0';
    len = 0;
}

// Free memory when destroying string object.
strtype::~strtype()
{
   cout << "Freeing p\n";
   free(p);
}

void strtype::set(char *ptr)
{
    if(strlen(p) >= SIZE)
    {
        cout << "String too big\n";
        return;
    }
    strcpy(p, ptr);
    len = strlen(p);
}

void strtype::show()
{
    cout << p << " - length: " << len;
    cout << "\n";
}

int main()
{
    strtype s1, s2;

    s1.set("This is a test.");
    s2.set("I like C++.");

    s1.show();
    s2.show();

    return 0;
}
```

The program uses **malloc** and **free()** to allocate and free memory. While this is perfectly valid, C++ does provide another way to dynamically manage memory, as you will see later in this book.

Note: *The preceding program uses the new-style headers for the C library functions used by the program. As mentioned in Chapter 1, if your compiler does not support these headers, simply substitute the standard C header files. This applies to other programs in this book in which C library functions are used.*

3. Here is an interesting way to use an object's constructor and destructor. This program uses an object of the **timer** class to time the interval between when an object of type **timer** is created and when it is destroyed. When the object's destructor is called, the elapsed time is displayed. You could use an object like this to time the duration of a program or the length of time a function spends within a block. Just make sure that the object goes out of scope at the point at which you want the timing interval to end.

```
#include <iostream>
#include <ctime>
using namespace std;

class timer
{
    clock_t start;
public:
    timer(); // constructor
    ~timer(); // destructor
};

timer::timer()
{
    start = clock();
}

timer::~timer()
{
    clock_t end;

    end = clock();
    cout << "Elapsed time: " << (end-start) / CLOCKS_PER_SEC
       << "\n";
}

int main()
{
    timer ob;
    char c;

    // delay ...
    cout << "Press a key followed by ENTER: ";
    cin >> c;

    return 0;
}
```

This program uses the standard library function **clock()**, which returns the number of clock cycles that have taken place since the program started running. Dividing this value by **CLOCKS_PER_SEC** converts the value to seconds.

EXERCISES

1. Rework the **queue** class that you developed as an exercise in Chapter 1 by replacing its initialization function with a constructor.

2. Create a class called **stopwatch** that emulates a stopwatch that keeps track of elapsed time. Use a constructor to initially set the elapsed time to 0. Provide two member functions called **start()** and **stop()** that turn on and off the timer, respectively. Include a member function called **show()** that displays the elapsed time. Also, have the destructor function automatically display elapsed time when a **stopwatch** object is destroyed. (To simplify, report the time in seconds.)

3. What is wrong with the constructor shown in the following fragment?

```
class sample
{
    double a, b, c;
public:
    double sample(); // error, why?
};
```

2.2 CONSTRUCTORS THAT TAKE PARAMETERS

It is possible to pass arguments to a constructor function. To allow this, simply add the appropriate parameters to the constructor function's declaration and definition. Then, when you declare an object, specify the arguments. To see how this is accomplished, let's begin with the short example shown here:

```
#include <iostream>
using namespace std;

class myclass
{
    int a;
public:
    myclass(int x);
    void show();
};

myclass::myclass(int x)
{
    cout << "In constructor\n";
    a = x;
}

void myclass::show()
{
    cout << a << "\n";
}

int main()
{
    myclass ob(4);

    ob.show();

    return 0;
```

```
}
```

Here the constructor for **myclass** takes one parameter. The value passed to **myclass()** is used to initialize **a**. Pay special attention to how **ob** is declared in **main()**. The value 4, specified in the parentheses following **ob**, is the argument that is passed to **myclass()**'s parameter **x**, which is used to initialize **a**.

Actually, the syntax for passing an argument to a parameterized constructor is shorthand for this longer form:

```
myclass ob = myclass(4)
```

however, most C++ programmers use the short form. Actually, there is a slight technical difference between the two forms that relates to copy constructors, which are discussed later in this book. But you don't need to worry about his distinction now.

Note: *Unlike constructor functions, destructor functions cannot have parameters. The reason for this simple enough to understand: there exists no mechanism by which to pass arguments to an object that is being destroyed.*

EXAMPLES

1. It is possible-in fact, quite common-to pass a constructor more than one argument. Here **myclass()** is passed two arguments:

```
#include <iostream>
using namespace std;

class myclass
{
    int a, b;
public:
    myclass(int x, int y); // constructor
    void show();
};

myclass::myclass(int x, int y)
{
    cout << "In constructor\n";
    a = x;
    b = y;
}

void myclass::show()
{
    cout << a << ' ' << b << "\n";
}

int main()
{
    myclass ob(4, 7);

    ob.show();

    return 0;
}
```

Here 4 is passed to **x** and 7 is passed to **y**. This same general approach is used to pass any number of arguments you like (up to the limit set by the compiler, of course).

2. Here is another version of the **stack** class that uses a parameterized constructor to pass a "name" to a stack. This single-character name is used to identify the stack that is being referred to when an error occurs.

```
#include <iostream>
using namespace std;

#define SIZE 10

// Declare a stack class for characters.
class stack
{
    char stck[SIZE]; // holds the stack
    int tos; // index of top of stack
    char who; // identifies stack
public:
    stack(char c); //constructor
    void push(char ch); // push character on stack
    char pop(); // pop character from stack
};

// Initialize the stack.
stack::stack(char c)
{
    tos = 0;
    who = c;
    cout << "Constructing stack " << who << "\n";
}

// Push a character.
void stack::push(char ch)
{
    if(tos==SIZE)
    {
        cout << "Stack " << who << " is full\n";
        return;
    }
    stck[tos] = ch;
    tos++;
}

// Pop a character.
char stack::pop()
{
    if(tos==0)
    {
        cout << "Stack " << who << " is empty\n";
        return 0; // return null on empty stack
    }
```

```
    tos--;
    return stck[tos];
}

int main()
{
    // Create two stacks that are automatically initialized.
    stack s1('A'), s2('B');
    int i;

    s1.push('a');
    s2.push('x');
    s1.push('b');
    s2.push('y');
    s1.push('c');
    s2.push('z');

    // This will generate some error messages.

    for(i=0; i<5; i++)
        cout << "Pop s1: " << s1.pop() << "\n";
    for(i=0; i<5; i++)
        cout << "Pop s2: " << s2.pop() << "\n";

    return 0;
}
```

Giving objects a "name," as shown in this example, is especially useful during debugging, when it is important to know which object generates an error.

3. Here is a different way to implement the **strtype** class (developed earlier) that uses a parameterized constructor function:

```
#include <iostream>
#include <cstring>
#include <cstdlib>
using namespace std;

class strtype
{
    char *p;
    int len;
public:
    strtype(char *ptr);
    ~strtype();
    void show();
};

strtype::strtype(char *ptr)
{
    len = strlen(ptr);
    p = (char *) malloc(len+1);
    if(!p)
```

```
    {
        cout << "Allocation error\n";
        exit(1);
    }
    strcpy(p, ptr);
}

strtype::~strtype()
{
   cout << "Freeing p\n";
   free(p);
}

void strtype::show()
{
    cout << p << " - length: " << len;
    cout << "\n";
}

int main()
{
    strtype s1("This is a test."), s2("I like C++.");

    s1.show();
    s2.show();

    return 0;
}
```

In this version of **strtype**, a string is given an initial value using the constructor function.

4. Although the previous examples have used constants, you can pass an object's constructor any valid expression, including variables. For example, this program uses user input to construct an object:

```
#include <iostream>
using namespace std;

class myclass
{
    int i, j;
public:
    myclass(int a, int b);
    void show();
};

myclass::myclass(int a, int b)
{
    cout << "In constructor\n";
    i = a;
    j = b;
}
```

```
void myclass::show()
{
     cout << i << ' ' << j << "\n";
}

int main()
{
     int x, y;

     cout << "Enter two integers: ";
     cin >> x >> y;

     // use variables to construct ob
     myclass ob(x, y);

     ob.show();

     return 0;
}
```

This program illustrates an important point about objects. They can be constructed as needed to fit the exact situation at the time of their creation. As you learn more about C++, you will see how useful constructing objects "on the fly" is.

EXERCISES

1. Change the **stack** class so it dynamically allocates memory for the stack. Have the size of the stack specified by a parameter to the constructor function. (Don't forget to free this memory with a destructor function.)

2. Create a class called **t_and_d** that is passed the current system time and date as a parameter to its constructor when it is created. Have the class include a member function that displays this time and date on the screen. (Hint: Use the standard time and date functions found in the standard library to find and display the time and date.)

3. Create a class called **box** whose constructor function is passed three **double** values, each of which represents the length of one side of a box. Have the **box** class compute the volume of the box and store the result in a **double** variable. Include a member function called **vol()** that displays the volume of each **box** object.

2.3 INTRODUCING INHERITANCE

Although inheritance is discussed more fully in Chapter 7, it needs to be introduced at this time. As it applies to C++, inheritance is the mechanism by which one class can inherit the properties of another. Inheritance allows a hierarchy of classes to be built, moving from the most general to the most specific.

To begin, it is necessary to define two terms commonly used when discussing inheritance. When one class is inherited by another, the class that is inherited is called the *base class*. The inheriting class is called the *derived class*. In general, the process of inheritance begins with the definition of a base class. The base class defines all qualities that will be common to any derived classes. In essence, the base class represents the most general description of a set of traits. A derived class inherits those general traits and adds properties that are specific to that class.

To understand how one class can inherit another, let's first begin with an example that, although simple, illustrates many key features of inheritance.
To start, here is the declaration for the base class:

```
// Define base class.
class B
{
    int i;
public:
    void set_i(int n);
    int get_i();
};
```

Using this base class, here is a derived class that inherits it:

```
// Define derived class.
class D : public B
{
    int j;
public:
    void set_j(int n);
    int mul();
};
```

Look closely at this declaration. Notice that after the class name **D** there is a colon followed by the keyword **public** and the class name **B**. This tells the compiler that class **D** will inherit all components of class **B**. They keyword **public** tells the compiler that **B** will be inherited such that all public elements of the base class will also be public elements of the derived class. However, all private elements of the base class remain private to it and are not directly accessible by the derived class.
Here is an entire program that uses the **B** and **D** classes:

```
// A simple example of inheritance.
#include <iostream>
using namespace std;

// Define base class.
class B
{
    int i;
public:
    void set_i(int n);
    int get_i();
};

// Define derived class.
class D : public B
{
    int j;
public:
    void set_j(int n);
    int mul();
};
```

```
// Set value i in base
void B::set_i)int n)
{
    i = n;
}

// Return value of i in base.
int B::get_i()
{
    return i;
}

// Set value of j in derived.
void D::set_j(int n)
{
    j = n
}

// Return value of base's i times derived's j.
int::mul()
{
    // derived class can call base class public member functions
    return j * get_i();
}

int main()
{
    D ob;

    ob.set_i(10); // load i in base
    ob.set_j(4); // load j in derived

    cout << ob.mul(); // displays 40

    return 0;
}
```

Look at the definition of **mul()**. Notice that it calls **get_i()**, which is a member of the base class **B**, not of **D**, without linking it to any specific object. This is possible because the public members of **B** become public members of **D**. However, the reason that **mul()** must call **get_i()** instead of accessing **i** directly is that the private members of a base class (in this case,**i**) remain private to it and not accessible by any derived class. The reason that private members of a class are not accessible to derived classes is to maintain encapsulation. If the private members of a class could be made public simply by inheriting the class, encapsulation could be easily circumvented.

The general form used to inherit a base class is shown here:

```
class derived_class_name : access_specifier base_class_name
```

Here *access-specifier* is one of the following three keywords: **public**, **private**, or **protected**. For now, just use **public** when inheriting a class. A complete description of the access specifiers will be given later in this book.

EXAMPLES

1. Here is a program that defines a generic base class called **fruit** that describes certain characteristics of fruit. This class is inherited by two derived classes **Apple** and **Orange**. These classes supply specific information to **fruit** that are related to these type of fruit.

```
// An example of class inheritance.
#include <iostream>
#include <cstring>
using namespace std;

enum yn {no, yes};
enum color {red, yellow, green, orange};

void out(enum yn x);

char *c[] = {"red", "yellow", "green", "orange"};

// Generic fruit class.
class fruit
{
// in this base, all elements are public
public:
    enum yn annual;
    enum yn perennial;
    enum yn tree;
    enum yn tropical;
    enum color clr;
    char name[40];
};

// Derive Apple class.
class Apple : public fruit
{
    enum yn cooking;
    enum yn crunchy;
    enum yn eating;
public:
    void seta(char *n, enum color c, enum yn ck, enum yn
       crchy, enum yn e);
    void show();
};

// Derive orange class.
class Orange : public fruit
{
    enum yn juice;
    enum yn sour;
    enum yn eating;
public:
    void seto(char *n, enum color c, enum yn j, enum yn sr,
       enum yn e);
    void show();
```

```
};

void Apple::seta(char *n, enum color c, enum yn ck, enum yn
   crchy, enum yn e)
{
    strcpy(name, n);
    annual = no;
    perennial = yes;

    tree = yes;
    tropical = no;
    clr = c;
    cooking = ck;
    crunchy = crchy;
    eating = e;
}

void Orange::seto(char *n, enum color c, enum yn j, enum yn
   sr, enum yn e)
{
    strcpy(name, n);
    annual = no;
    perennial = yes;
    tree = yes;
    tropical = yes;
    clr = c;
    juice = j;
    sour = sr;
    eating = e;
}

void Apple::show()
{
    cout << name << " apple is: "<< "\n";
    cout << "Annual: "; out(annual);
    cout << "Perennial: "; out(perennial);
    cout << "Tree: "; out(tree);
    cout << "Tropical: "; out(tropical);
    cout << "Color: " << c[clr] << "\n";
    cout << "Good for cooking: "; out(cooking);
    cout << "Crunchy: "; out(crunchy);
    cout << "Good for eating: "; out(eating);
    cout << "\n";
}

void Orange:: show()
{
    cout << name << " orange is: " << "\n";
    cout << "Annual: "; out(annual);
    cout << "Perennial: "; out(perennial);
    cout << "Tree: "; out(tree);
```

```
    cout << "Tropical: "; out(tropical);
    cout << "Color: " << c[clr] << "\n";
    cout << "Good for juice: "; out(juice);
    cout << "Sour: "; out(juice);
    cout << "Good for eating: "; out(eating);
    cout << "\n";
}

void out(enum yn x)
{
    if(x==no)
        cout << "no\n";
    else
        cout << "yes\n";
}

int main()
{
    Apple a1, a2;
    Orange o1, o2;

    a1.seta("Red Delicious", red, no, yes, yes);
    a2.seta("Jonathan", red, yes, no, yes);

    o1.seto("Navel", orange, no, no, yes);
    o2.seto("Valencia", orange, yes, yes, no);

    a1.show();
    a2.show();

    o1.show();
    o2.show();

    return 0;
}
```

As you can see, the base class **fruit** defines several characteristics that are common to all types of fruit. (Of course, in order to keep this example short enough to fit conveniently in a book, the **fruit** class is somewhat simplified.) For example, all fruit grows on either annual or perennial plants, such as vines or bushes. All fruit has a color and a name. This base class is then inherited by the **Apple** and **Orange** classes. Each of these classes supplies information specific to its type of fruit.

This example illustrates the basic reason for inheritance. Here, a base class is created that defines the general traits associated with *all* fruit. It is left to the derived classes to supply those traits that are specific to each *individual* case.

This program illustrates another important fact about inheritance. A base class is not exclusively "owned" by a derived class. A base class can be inherited by any number of classes.

EXERCISE

1. Given the following base class,

```
class area_cl
{
public:
    double height;
    double width;
};
```

create two derived classes called **rectangle** and **isosceles** that inherit **area_cl**. Have each class include a function called **area()** that returns the area of a rectangle or isosceles triangle, as appropriate. Use parameterized constructors to initialize **height** and **width**.

2.4 OBJECT POINTERS

So far, you have been accessing members of an object by using the dot operator, This is the correct method when you are working with an object. However, it is also possible to access a member of an object via pointer to that object. When a pointer is used, the arrow operator (->) rather than the dot operator is employed. (This is exactly the same way the arrow operator is used when given a pointer to a structure.)
You declare an object pointer just like you declare a pointer to any other variable. Specify its class name, and then precede the variable name with an asterisk. To obtain the address of an object, precede the object with **&** operator, just as you do when taking the address of any other type of variable.
Just like pointers to other types, an object pointer, when incremented, will point to the next object of its type.

EXAMPLE

1. Here is a simple example that uses an object pointer:

```
#include <iostream>
using namespace std;

class myclass
{
    int a;
public:
    myclass(int x); // constructor
    int get();
};

myclass::myclass(int x)
{
    a = x;
}

int myclass::get()
{
    return a;
}

int main()
{
```

```
    myclass ob(120); // create object
    myclass *p; // create pointer to object

    p = &ob; // put address of ob into p

    cout << "Value using object: " << ob.get();
    cout << "\n";

    cout << "Value using pointer: " << p->get();

    return 0;
}
```

Notice how the declaration

```
myclass *p;
```

creates a pointer to an object of **myclass**. It is important to understand that creation of an object pointer does *not* create an object-it creates just a pointer to one. The address of **ob** is put into **p** by using this statement:

```
p = &ob;
```

Finally, the program shows how the members of an object can be accessed through a pointer.

We will come back to the subject of object pointers in Chapter 4, once you know more about C++. There are several special features that relate to them.

2.5 CLASSES, STRUCTURES, AND UNIONS ARE RELATED

As you have seen, the class is syntactically similar to the structure. You might be surprised to learn, however, that the class and the structure have virtually identical capabilities. In C++, the definition of structure has been expanded so that it can also include member functions, including constructor and destructor functions, in just the same way that a class can. In fact, the only difference between a structure and a class is that, by default, the members of a class are private but the members of a structure are public. The expanded syntax of a structure is shown here:

```
struct type_name
{
    // public function and data members
private:
    // private function and data members
} object_list;
```

In fact, according to the formal C++ syntax, both **struct** and **class** create new class types. Notice that a new keyword is introduced. It is **private**, and it tells the compiler that follow are private to that class.

On the surface, there is a seeming redundancy in the fact that structures and classes have virtually identical capabilities. Many newcomers to C++ wonder why this apparent duplication exists. In fact, it is not uncommon to hear the suggestion that the **class** keyword is unnecessary. The answer to this line of reasoning has both a "strong" and "weak" form. The "strong" (or compelling) reason concerns maintaining upward compatibility from C. In C++, a C-style structure is also perfectly acceptable in a C++ program. Since in C all structure members

are public by default, this convention is also maintained in C++. Further, because **class** is a syntactically separate entity from **struct**, the definition of a class is free to evolve in a way that will not be compatible with a C-like structure definition. Since the two are separated, the future direction of C++ is not restricted by compatibility concerns.

The "weak" reason for having two similar constructs is that there is no disadvantage to expanding the definition of a structure in C++ to including member functions.

Although structures have the same capabilities as classes, most programmers restrict their use of structures to adhere to their C-like form and do not use them to include function members. Most programmers use the **class** keyword when defining objects that contain both data and code. However, this is a stylistic matter and is subject to your own preference. (After this, section, this book reserves the use of **struct** for objects that have no function members.)

If you find the connection between classes and structures interesting, so will you find this next revelation about C++: unions and classes are also related. In C++, a union defines a class type that can contain both functions and data as members. A union is like a structure in that, by default, all members are public until the **private** specifier is used. In a union, however, all data members share the same memory location (just as in C). Unions can contain constructor and destructor functions. Fortunately, C unions are upwardly compatible with C++ unions.

Although structures and classes seem on the surface to be redundant, this is not the case with unions. In an object-oriented language, it is important to preserve encapsulation. Thus, the union's ability to link code and data allows you to create class types in which all data uses a shared location. This is something that you cannot do using a class.

There are several restrictions that apply to unions as they relate to C++. First, they cannot inherit any other class and they cannot be used as a base class for any other type. Unions must not have any **static** members. They also must not contain any object that has a constructor or destructor. The union, itself, *can* have a constructor and destructor though. Finally, unions cannot have virtual member functions. (Virtual functions are described later in this book.)

There is a special type of union in C++ called an *anonymous union*. An anonymous union does not have a type name, and no variables can be declared for this sort of union. Instead, an anonymous union tells the compiler that its members will share the same memory location. However, in all other, respects, the members act and are treated like normal variables. That is, the members are accessed directly, without the dot operator syntax. For example, examine this fragment:

```
union // an anonymous union
{
     int i;
     char ch[4];
};

i = 10; //  access i and ch directly
ch[0] = 'X';
```

As you can see, **i** and **ch** are accessed directly because they are not part of any object. They do, however, share the same memory space.

The reason for the anonymous union is that it gives you a simple way to tell the compiler that you want two or more variables to share the same memory location. Aside from this special attribute, members of an anonymous union behave like other variables.

Anonymous unions have all of the restrictions that apply to normal unions, plus these additions. A global anonymous union must be declared **static**. An anonymous union cannot contain private members, The names of the members of an anonymous union must not conflict with other identifiers within the same scope.

EXAMPLES

1. Here is a short program that uses **struct** to create a class:

```
#include <iostream>
#include <cstring>
using namespace std;

// use struct to define a class type
struct st_type
{
    st_type(double b, char *n);
    void show();
private:
    double balance;
    char name[40];
};

st_type::st_type(double b, char *n)
{
    balance = b;
    strcpy(name, n);
}

void st_type::show()
{
    cout << "Name: " << name;
    cout << ": $" << balance;
    if(balance<0.0)
        cout << "**";
    cout << "\n";
}

int main()
{
    st_type acc1(100.12, "Johnson");
    st_type acc2(-12.34, "Hedricks");

    acc1.show();
    acc2.show();

    return 0;
}
```

Notice that, as stated, the members of a structure are public by default. The **private** keyword must be used to declare private members.

Also, notice one difference between C-like structures and C++-like structures. In C++, the structure tag-name also becomes a complete type name that can be used to declare objects. In C, the tag-name requires that the keyword **struct** precede it before it becomes a complete type.

Here is the same program, rewritten using a class:

```
#include <iostraem>
```

```
#include <cstring>
using namespace std;

class cl_type
{
    double balance;
    char name[40];
public:
    cl_type(double b, char *n);
    void show();

};

cl_type::cl_type(double b, char *n)
{
    balance = b;
    strcpy(name, n);
}

void cl_type::show()
{
    cout << "Name: " << name;
    cout << ": $" << balance;
    if(balance<0.0)
        cout << "**";
    cout << "\n";
}

int main()
{
    cl_type acc1(100.12, "Johnson");
    cl_type acc2(-12.34, "Hedricks");

    acc1.show();
    acc2.show();

    return 0;
}
```

2. Here is an example that used a union to display the binary bit pattern, byte by byte, contained within a **double** value.

```
#include <iostream>
using namespace std;

union bits
{
    bits(double n);
    void show_bits();
    double d;
    unsigned char c[sizeof(double)];
};
```

```
bits::bits(double n)
{
    d = n;
}

void bits::show_bits()
{
    int i, j;

    for(j = sizeof(double)-1; j>=0; j--)
    {
        cout << "Bit pattern in byte " << j << ": ";
        for(i = 128; i; i >>= 1)
            if(i & c[j])
                cout << "1";
            else
                cout << "0";
        cout << "\n";
    }
}

int main()
{
    bits ob(1991.829);

    ob.show_bits();

    return 0;
}
```

The output of this program is:

```
Bit pattern in byte 7:  01000000
Bit pattern in byte 6:  10011111
Bit pattern in byte 5:  00011111
Bit pattern in byte 4:  01010000
Bit pattern in byte 3:  11100101
Bit pattern in byte 2:  01100000
Bit pattern in byte 1:  01000001
Bit pattern in byte 0:  10001001
```

3. Both structures and unions can have constructors and destructors. The following example shows the **strtype** class reworked as a structure. It contains both a constructor and a destructor function.

```
#include <iostream>
#include <cstring>
#include <cstdlib>
```

```
using namespace std;

struct strtype
{
    strtype(char *ptr);
    ~strtype();
    void show();
private:
    char *p;
    int len;
};

strype::strype(char *ptr)
{
    len = strlen(ptr);
    p = (char *)malloc(len+1);
    if(!p)
    {
        cout << "Allocation error\n";
        exit(1);
    }
    strcpy(p, ptr);
}

strtype::~strtype()
{
    cout << "Freeing p\n";
    free(p);
}

void strtype::show()
{
    cout << p << " - length: " << len;
    cout << "\n";
}

int main()
{
    strtype s1("This is a test."), s2("I like C++");

    s1.show();
    s2.show();

    return 0;
}
```

This program uses an anonymous union to display the individual bytes that comprise a **double**. (This program assumes that **double**s are 8 bytes long.)

```
// Using an anonymous union.
#include <iostream>
using namespace std;
```

```
int main()
{
    union
    {
        unsigned char bytes[8];
        double value;
    };
    int i;
    value = 859345.324;

    // display the bytes within a double
    for(i=0; i<8; i++)
        cout << (int)bytes[i] << " ";

    return 0;
}
```

As you can see, both **value** and **bytes** are accessed as if they were normal variables and not part of a union. Even though they are declared as being part of an anonymous union, their names are at the same scope level as any other local variable declared at the same point. This is why a member of an anonymous union cannot have the same name as any other variable known to its scope.

EXERCISES

1. Rewrite the **stack** class presented in Section 2.1 so it uses a structure rather than a class.
2. Use a **union** class to swap the low- and high-order bytes of an integer (assuming 16-bit integers; if your computer uses 32-bit integers, swap the bytes of a **short int**).
3. Explain what an anonymous union is and how it differs from a normal union.

2.6 IN-LINE FUNCTIONS

Before we continue this examination of classes, a short but related digression is needed. In C++, it is possible to define functions that are not actually called but, rather, are expanded in line, at the point of each call. This is much the same way that a C-like parameterized macro works. The advantage if *in-line functions* is that they have no overhead associated with the function call and return mechanism. This means that in-line functions can be executed much faster than normal functions. (Remember, the machine instructions that generate the function call and return take time each time a function is called. If there are parameters, even more time overhead is generated.)
The disadvantages of in-line function is that if they are too large and called too often, your program grows larger. For this reason, in general only short functions are declared as in-line functions.
To declare an in-line function, simply precede the function's definition with the **inline** specifier. For example, this short program shows how to declare an in-line function:

```
// Example of an in-line function
#include <iostream>
using namespace std;
```

```
inline int even(int x)
{
    return !(x%2);
}

int main()
{
    if(even(10))
        cout << "10 is even\n";
    if(even(11))
        cout << "11 is even\n";

    return 0;
|
```

In this example, the function **even()**, which returns **true** if its argument is even, is declared as being-in-line. This means that the line

```
if(even(10))
        cout << "10 is even\n";
```

is functionally equivalent to:

```
if(!(10@2))
        cout << "10 is even\n";
```

This example also points out another important feature of using **inline**: an in-line function must be defined *before* it is first called. If it isn't, the compiler has no way to know that it is supposed to be expanded in-line. This is why **even()** was defined before **main()**.

The advantages of using **inline** rather than parameterized macros is twofold. First, it provides a more structured way to expand short functions in line. For example, when you are creating a parameterized macro, it is easy to forget that extra parentheses are often needed to ensure proper in-line expansion in every case. Using in-line functions prevents such problems.

Second, an in-line function might be able to be optimized more thoroughly by the compiler than a macro expansion. In any event, C++ programmers virtually never use parameterized macros, instead relying on **inline** to avoid the overhead of a function call associated with a short function.

It is important to understand that the **inline** specifier is a *request*, not a command, to the compiler. If, for various reason, the compiler is unable to fulfill the request, the function is compiled as a normal function and the **inline** request is ignored.

Depending upon your compiler, several restrictions to in-line functions may apply. For example, some compilers will not in-line a function if it contains a **static** variable, a loop statement, a **switch** or a **goto**, or if the function is recursive. You should check your compiler's user manual for specific restrictions to in-line functions that might affect you.

Remember: *If any in-line restriction is violated, the compiler is free to generate a normal function.*

EXAMPLES

1. Any type of function can be in-lined, including functions that are member of classes. For example, here the member function **divisible()** is in-lined for fast execution. (The function returns **true** if its first argument can be evenly divided by its second.)

```
// Demonstrate in-lining a member function.
#include <iostream>
using namespace std;

class samp
{
    int i, j;
public:
    samp(int a, int b);
    int divisible(); // in-lined in its definition
};

samp::samp(int a, int b)
{
    i = a;
    j = b;
}

/*
   Return 1 if i is evenly divisible by j.
   This member function is expanded in line.
*/
inline int samp::divisible()
{
    return !(i%j);
}

int main()
{
    samp ob1(10, 2), ob2(10, 3);

    // this is true
    if(ob1.divisible())
        cout << "10 divisible by 2\n";

    // this is false
    if(ob2.divisible())
        cout << "10 divisible by 3\n";

    return 0;
}
```

2. It is perfectly permissible to in-line an overloaded function. For example, this program overloads **min()** three ways. Each way is also declared as **inline**.

```
#include <iostream>
using namespace std;

// Overload min() three ways.

//integers
inline int min(int a, int b)
```

```
{
    return a<b ? a : b;
}

//longs
inline long min(long a, long b)
{
    return a<b ? a : b;
}

//doubles
inline double min(double a, double b)
{
    return a<b ? a : b;
}

int main()
{
    cout << min(-10, 10) << "\n";
    cout << min(-10.0, 100.002) << "\n";
    cout << min(-10L, 12L) << "\n";

    return 0;
}
```

EXERCISES

1. In Chapter 1 you overloaded the **abs()** function so that it could find the absolute value of integers, long integers, and **doubles**. Modify that program so that those functions are expanded in line.

2. Why might the following function not be in-lined by your compiler?

```
void f1()
{
    int i;

    for(i=0; i<10; i++)
        cout << i;
}
```

2.7 AUTOMATIC IN-LINING

If a member function's definition is short enough, the definition can be included inside the class declaration. Doing so causes the function to automatically become an in-line function, if possible. When a function is defined within a class declaration, the **inline** keyword is no longer necessary. (However, it is not an error to use it in this situation.) For example, the **divisible()** function from the preceding section can be automatically in-lined as shown here:

```
#include <iostream>
using namespace std;
```

```
class samp
{
    int i, j;
public:
    samp(int a, int b);

    /*
       divisible() is defined here and automatically
       in-lined.
    */
    int divisible() { return !(i%j);}
};

samp::samp(int a, int b)
{
    i = a;
    j = b;
}

int main()
{
    samp ob1(10, 2), ob2(10, 3);

    // this is true
    if(ob1.divisible())
        cout << "10 divisible by 2\n";

    // this is false
    if(ob2.divisible())
        cout << "10 divisible by 3\n";

    return 0;
}
```

As you can see, the code associated with **divisible()** occurs inside the declaration for the class **samp**. Further notice that no other definition of **divisible()** is needed-or permitted. Defining **divisible()** inside **samp** causes it to be made into an in-line function automatically.

When a function defined inside a class declaration cannot be made into an in-line function (because a restriction has been violated), it is automatically made into a regular function.

Notice how **divisible()** is defined within **samp**, paying particular attention to the body. It occurs all on one line. This format is very common in C++ programs when a function is declared within a class declaration. It allows the declaration to be more compact. However, the **samp** class could have been written like this:

```
class samp
{
    int i, j;
public:
    samp(int a, int b);

    /*
       divisible() is defined here and automatically in-lined.
    */
```

```
    int divisible()
    {
        return !(i%j);
    }
};
```

In this version, the layout of **divisible()** uses the more-or-less standard indentation style. From the compiler's point of view, there is no difference between the compact style is commonly found in C++ programs when short functions are defined inside a class definition.
The same restrictions that apply to "normal" in-line functions apply to automatic in-line functions within a class declaration.

EXAMPLES

1. Perhaps the most common use of in-line functions defined within a class is to define constructor and destructor functions. For example, the **samp**class can more efficiently be defined like this:

```
#include <iostream>
using namespace std;

class samp
{
    int i, j;
public:
    // inline constructor
    samp(int a, int b) { i = a; j = b; }
    int divisible() { return !(i%j); }
};
```

The definition of **samp()** within the class **samp** is sufficient, and no other definition of **samp()** is needed.

2. Sometimes a short function will be included in a class declaration even though the automatic in-lining feature is of little or no value. Consider this class declaration:

```
class myclass
{
    int i;
public:
    myclass(int n) { i = n; }
    void show() { cout << i; }
};
```

Here the function **show()** is made into an in-line function automatically. However, as you should know, I/O operations are (generally) so slow relative to CPU/memory operations that any effect of eliminating the function call overhead is essentially lost. Even so, in C++ programs, it is still common to see small functions of this type declared within a class simply for the sake of convenience, and because no harm is caused.

EXERCISES

1. Convert the **stack** class from Section 2.1, Example 1, so that it uses automatic in-line functions where appropriate.

2. Convert the **strtype** class from Section 2.2, Example 3, so that it uses automatic in-line functions.

SKILLS CHECK

Mastery Skills Check

At this point you should be able to perform the following exercises and answer the questions.

1. What is a constructor? What is a destructor? When are they executed?

2. Create a class called **line** that draws a line on the screen. Store the line length in a private integer variable called **len**. Have **line**'s constructor take one parameter: the line length. Have the constructor store the length and actually draw the line. If your system does not support graphics, display the line by using *.
   Optional: Give **line**a destructor that erases the line.

3. What does the following program display?

```
#include <iostream>
using namespace std;

int main()
{
    int i = 10;
    long l = 1000000;
    double d = -0.0009;

    cout << i << ' ' << l << ' ' << d;
    cout << "\n";

    return 0;
}
```

4. Add another derived class that inherits **area_cl** from Section 2.3, Exercise 1. Call this class **cylinder** and have it compute the surface area of a cylinder. Hint: The surface area of a cylinder is: $2 * \pi * R^2 + \pi * D * height$.

5. What is an in-line function? What are its advantages and disadvantages?

6. Modify the following program so that all member functions are automatically in-lined:

```
#include <iostream>
using namespace std;

class myclass
{
    int i, j;
public:
    myclass(int x, int y);
    void show();
};

myclass::myclass(int x, int y)
```

```
{
    i = x;
    j = y;
}

void myclass::show()
{
    cout << i << " " << j << "\n";
}

int main()
{
    myclass count(2, 3);

    count.show();

    return 0;
}
```

7. What is the difference between a class and a structure?

8. Is the following fragment valid?

```
union
{
    float f;
    unsigned int bits;
};
```

Cumulative Skills Check

This section checks how well you have integrated material in this chapter with that from the preceding chapter.

1. Create a class called **prompt**. Pass its constructor function a prompting string of your own choosing. Have the constructor display the string and then input an integer. Have the constructor display the string and then input an integer. Store this value in a private variable called **count**. When an object of type **prompt** is destroyed, ring the bell on the terminal as many times as the user entered.

2. In Chapter 1 you created a program that converted feet to inches. Now create a class that does the same thing. Have the class store the number of feet and its equivalent number of inches. Pass to the class's constructor the number of feet and have the constructor display the number of inches.

3. Create a class called **dice** that contains one private integer variable. Create a function called **roll()** that uses the standard random number generator, **rand()**, to generate a number between 1 and 6. Then have **roll()** display that value.